A3 Report: Tiny Transformer for Shakespeare Next-Token Prediction

Yitong Bai yb2636

repo: https://github.com/BrianBai093/Applied-ML/blob/main/A3/A3_Tiny_Transformer_Starter.ipynb

1. Objective

The goal of this assignment was to understand the Transformer architecture by implementing a small causal language model for next-token prediction on Tiny Shakespeare. The model was built from scratch in PyTorch, including positional encoding, self-attention, causal masking, residual connections, RMSNorm, and the feed-forward network. Beyond implementation, the report focuses on three outcomes: whether the model trains successfully, how learning rate and model size affect performance, and what the attention maps reveal about the model's behavior.

2. Dataset and Tokenization

The dataset was the Tiny Shakespeare corpus, containing 1,115,394 characters of play-style dialogue. Because the text includes speaker names, line breaks, and punctuation, it is a useful setting for studying both prediction quality and attention structure.

A custom subword-level BPE tokenizer was trained with a vocabulary size of 500, matching the assignment constraint. The full corpus was encoded into 641,904 tokens, giving a compression ratio of 1.74 characters per token. The token stream was then split into overlapping sequences of length 64, where the input is the first 64 tokens and the target is the same sequence shifted by one position. Following the assignment instructions, 80% of the tokens were used for training and 20% for validation, with no separate test set.

Key dataset statistics: corpus size 1,115,394 characters; vocabulary size 500; encoded corpus length 641,904 tokens; context length 64 tokens; training tokens 513,523; validation tokens 128,381.

3. Model Architecture

The model is a small causal Transformer language model. Input token IDs are mapped with `nn.Embedding`, combined with sinusoidal positional encodings, processed by two Transformer blocks, and projected back to the vocabulary to predict the next token. Each block uses RMSNorm, causal multi-head self-attention, residual connections, and a feed-forward network.

The final configuration used 2 Transformer blocks, hidden size 64, 4 attention heads, 4x feed-forward expansion, dropout 0.10, sinusoidal positional encoding, RMSNorm, and 164,276 trainable parameters.

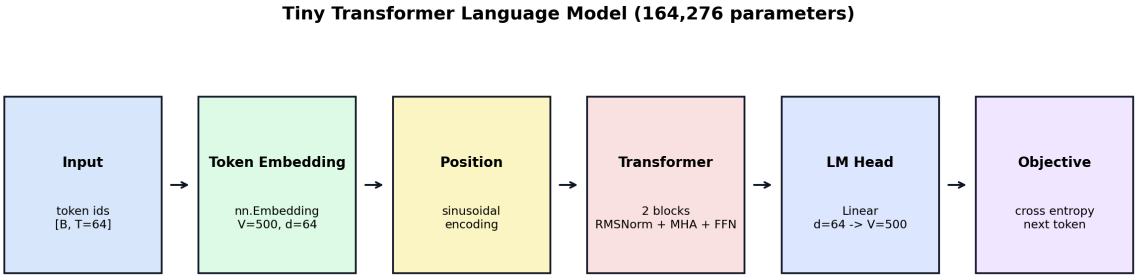

Figure 1. Visual summary of the Tiny Transformer language model.

The causal mask is critical: each token can attend only to itself and earlier tokens, which keeps the task autoregressive and prevents future-token leakage.

4. Training Setup

The model was trained with cross-entropy loss over all positions, and validation perplexity was computed as `exp(validation loss)`. AdamW with weight decay was used as the optimizer. To keep the assignment lightweight, each epoch used a limited number of sampled training batches instead of a full pass over every overlapping sequence.

Item	Value
Task	Next-token prediction
Dataset split	80% train / 19% validation
Vocabulary size	500
Context length	64
Batch size	32
Optimizer	AdamW
Learning rate	3.0e-04
Weight decay	0.01
Loss	Cross-entropy
Gradient clip	1.0
Epochs	5
Device	cuda
Seed	42

Figure 2. Training configuration used for the main experiment.

The main run used a learning rate of `3e-4`, batch size 32, context length 64, and five epochs. Training ran on CUDA and finished in 5.8 seconds, with random seeds set for reproducibility.

5. Main Training Results

The model learned steadily across five epochs. Training loss fell from 4.9081 to 3.6103, validation loss fell from 4.2864 to 3.5760, and the final validation perplexity reached 35.73.

Per-epoch metrics (train loss / validation loss / validation PPL) were: E1 4.9081 / 4.2864 / 72.70; E2 4.1931 / 3.9886 / 53.98; E3 3.9235 / 3.7818 / 43.90; E4 3.7314 / 3.6629 / 38.97; E5 3.6103 / 3.5760 / 35.73.

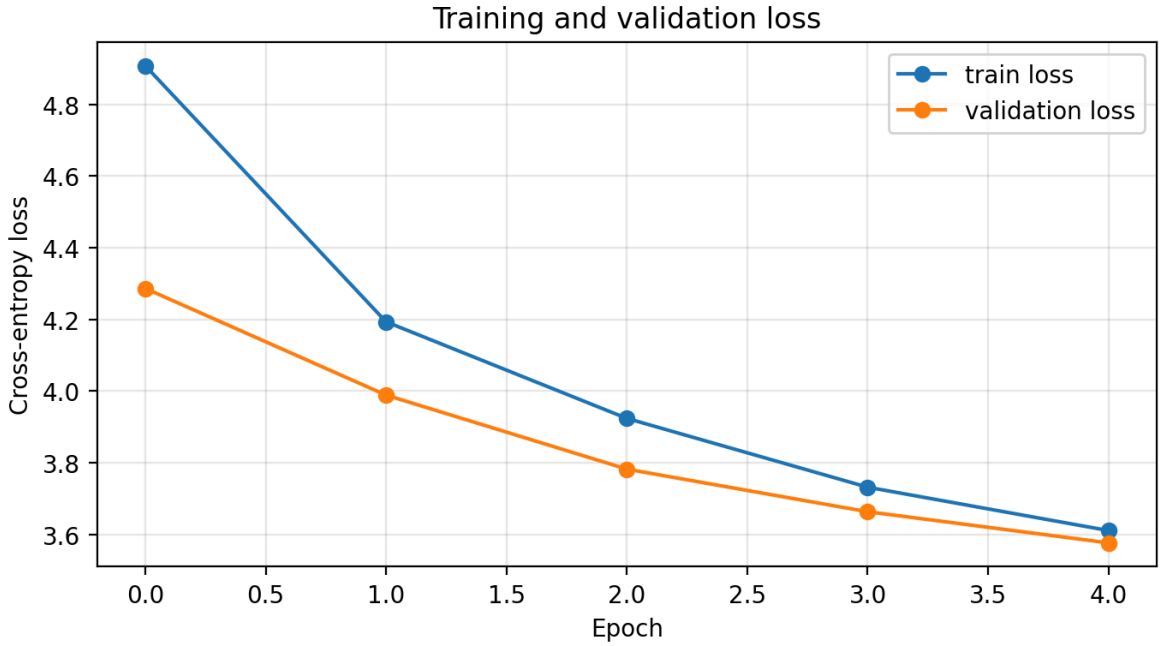

Figure 3. Training and validation cross-entropy loss over five epochs.

The validation curve follows the same downward trend as the training curve, suggesting that the model learned useful structure rather than only memorizing sampled batches. Validation loss remained slightly below training loss, which is reasonable because dropout is active during training but disabled during evaluation.

6. Hyperparameter Experiments

Two short controlled experiments were run: a learning rate comparison and a model size comparison. Each used the same lightweight budget of three epochs and 80 training batches per epoch, so the results should be read as relative comparisons rather than fully tuned final runs.

6.1 Learning Rate

The learning rate comparison tested `1e-3`, `3e-4`, and `1e-4` with the same model size. Under this short budget, `1e-3` performed best with validation loss 3.6271 and PPL 37.60, while lower learning rates improved more slowly (`3e-4` : PPL 57.94, `1e-4` : PPL 87.47).

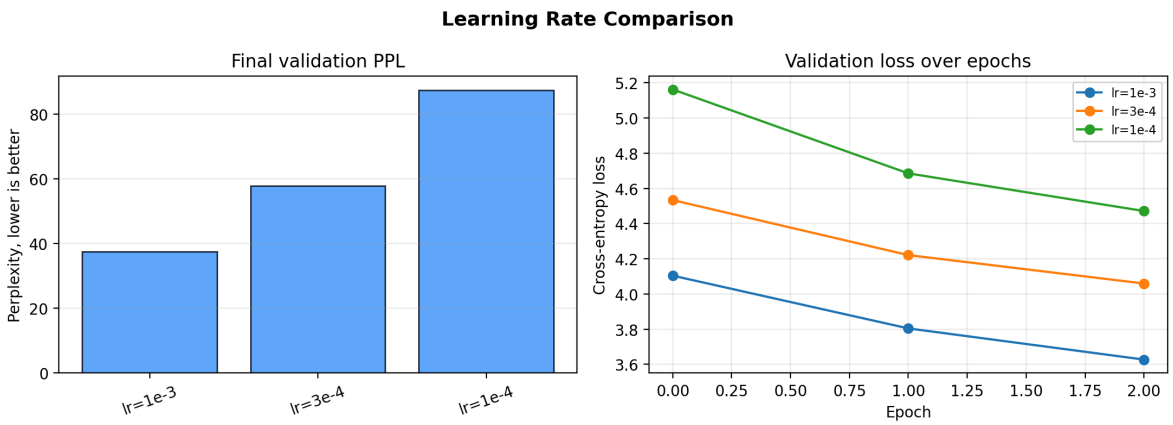

Figure 4. Learning rate comparison under a fixed short training budget.

These results suggest that the lower learning rates were simply too slow under the short experiment budget. The `1e-3` setting converged fastest without obvious instability over three epochs, although a longer run could still favor a more conservative value.

6.2 Model Size

The model size comparison tested hidden sizes 64 and 128 while keeping two Transformer blocks. Increasing the hidden size raised the parameter count from 164,276 to 524,660 and improved validation loss from 4.0594 to 3.7261, reducing perplexity from 57.94 to 41.52.

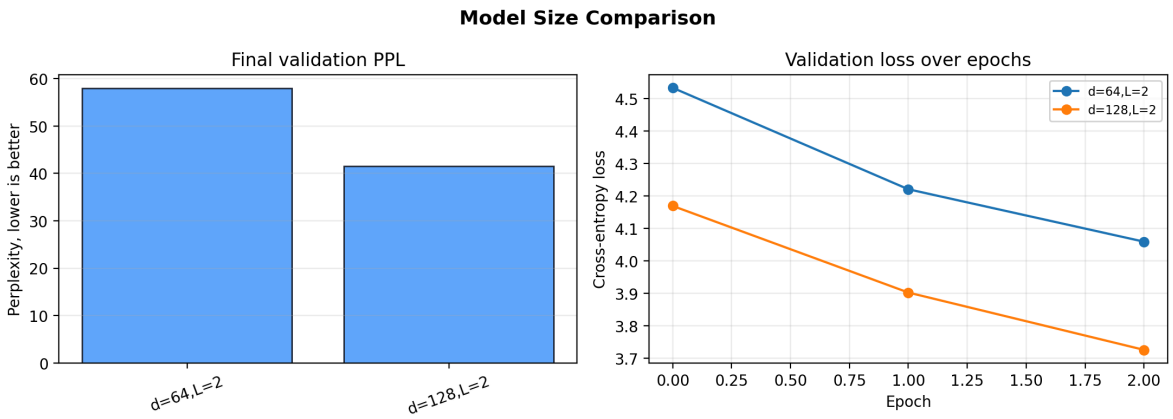

Figure 5. Model size comparison under a fixed short training budget.

This shows that model capacity mattered for this task. The larger model fit the token patterns better, though at a higher memory and compute cost.

7. Attention Visualization and Interpretation

The attention visualizations were computed during evaluation with `torch.no_grad()`. In each heatmap, query tokens are on the vertical axis and key tokens are on the horizontal axis. The dark upper-right triangle reflects the causal mask, which blocks attention to future positions.

The decoded validation sequence used for the attention visualizations was:

> "Sign me a present pardon for my brother, / Or with an outst..."

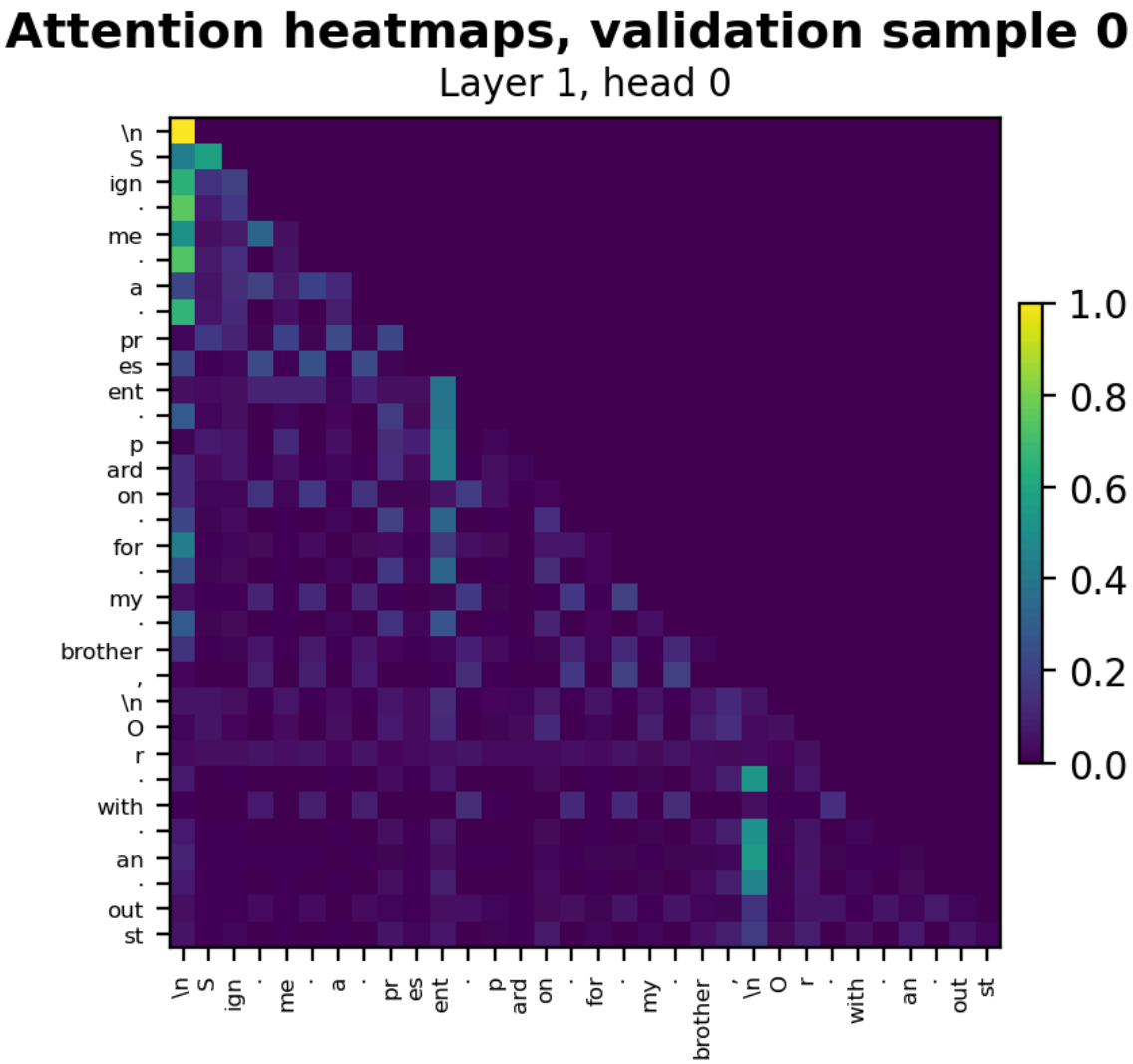

Figure 6. Close-up of the final layer, head 0 attention pattern.

The close-up heatmap confirms correct causal behavior: most attention mass lies in the lower triangle, and a visible diagonal shows that many tokens rely strongly on recent context.

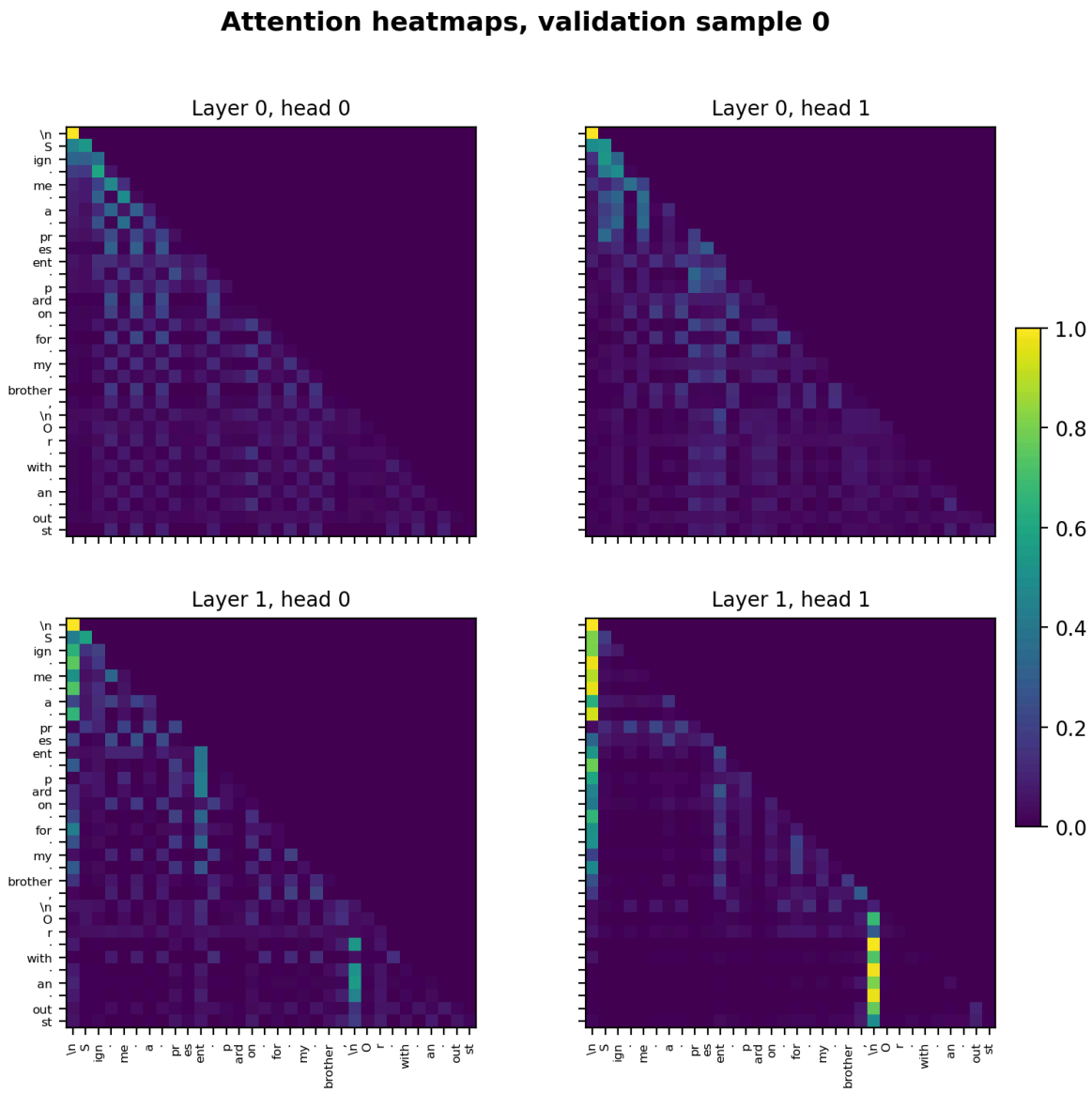

Figure 7. Comparison across layers and attention heads for the same validation sequence.

The layer/head comparison shows that the heads are not redundant. Some focus on local context near the diagonal, while others form stronger vertical bands at specific earlier tokens. In this sample, line breaks, punctuation, and clause-boundary tokens receive concentrated attention.

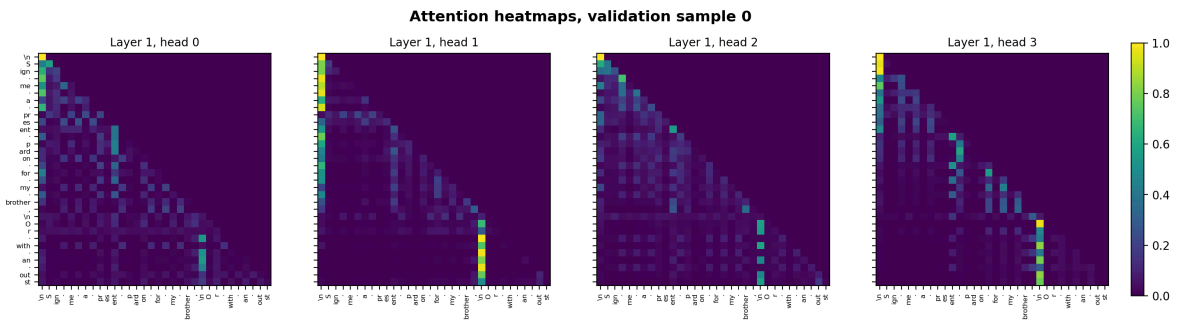

Figure 8. Final-layer attention heads on a structured validation sequence.

The final-layer heads also show different roles. Head 0 stays relatively distributed, while other heads focus more sharply on structural markers such as newlines and phrase boundaries. Even this small model appears to divide attention across different contextual functions.

8. Qualitative Generation

A short generation sample from the prompt “First Citizen:” showed some word-like fragments and occasional Shakespeare-style formatting, but it was not fluent. This suggests the model learned local token statistics and formatting patterns, yet still lacked enough long-range structure for coherent dialogue.

9. Discussion and Reflection

The most important implementation detail was the causal mask; without it, the model would leak future information and the task would no longer be valid. Positional encoding was also necessary because self-attention alone does not encode token order. The hyperparameter experiments showed that both learning rate and model capacity affected performance, with `1e-3` learning fastest in the short comparison and the larger hidden size performing better overall.

The main runtime bottleneck remains self-attention, whose cost grows quadratically with context length. In this assignment the context length was only 64, so training stayed fast. The final validation PPL of 35.73 is reasonable for a small model trained briefly on a limited corpus: it learned local language patterns and some structure, but not enough for strong long-form generation.

10. Conclusion

This assignment covered the full pipeline of a causal Transformer language model, from subword tokenization to perplexity evaluation and attention analysis. The model trained successfully, reaching a validation loss of 3.5760 and a validation PPL of 35.73. The attention plots showed correct causal behavior and meaningful differences across heads, while the experiments showed that both learning rate and hidden size affected results. Overall, the project demonstrates that even a small Transformer already captures the main ideas behind modern autoregressive language models.

AI Tool Usage Disclosure

I used AI assistance to help organize the report, and improve clarity. And asked AI to give me a frame with TODO, then implement it by myself while asking AI about how to use certain function or explain the code. And use AI todo the code review and debug.